Genetic CNNs to Optimize Kākā Identification

Alex Welsby
Victoria University of Wellington
welsbyalex@myvuw.ac.nz

I. Introduction

Marking individual animals is necessary for ecological research. However, at high population densities, traditional marking methods such as bird banding incur increasingly high costs and can fail to capture a holistic picture of the population's trends.

Recent developments in artificial intelligence have resulted in the development of novel machine learning applications for ecological studies. The Recognizing Taonga with AI project at Victoria University of Wellington aims to use AI vision as an alternative to traditional marking methods for taonga species, with a current focus on kākā. Thusfar, SIFT [1] and ViT-S [2] have been explored as potential methods for identifying kākā. I propose a new approach to identifying kākā individuals following Ferreira et al. 2020's pipeline for identifying individual birds [3] utilising 1). an instance segmentation step to mask the background of a bird image out, 2). pre-processing of the masked images to aid generalization in the classifier, 3). and a final Convolutional Neural Network (CNN) classifier, developed through Evolutionary Neural Algorithm Search (ENAS) methods, that identifies individual bird classes.

II. The Dataset

The kākā dataset is currently composed of three custom datasets taken in Zealandia, labelled A ($n = 7$; where n=number of labeled individuals), B ($n = 51$), and C ($n = 32$). All three datasets comprise of still frames of kākā attempting to use a feeder nozzle against a plain white background; a newer camera was used for Dataset C. Each dataset provides labels to its birds that correspond to their unique combination of leg bands, such as Y-MY (Yellow-MauveYellow) or O-RS (Orange-RedSilver).

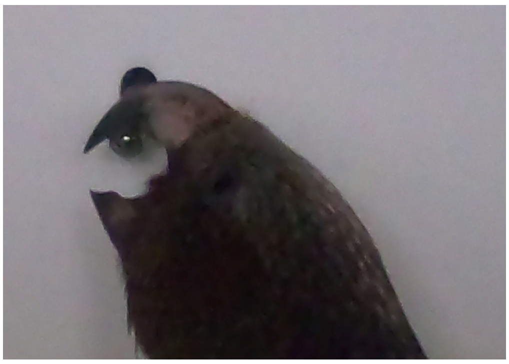

Fig. 1. An example photo taken from Dataset B. Note how the beak overlaps with the feeder nozzle, which are both similar in pixel intensity.

Dataset A was gathered in 2021 [1], while datasets B and C were gathered several weeks apart in 2022 [4].

This means that each dataset represents a temporal transformation on the birds and their features, with the largest temporal difference between Dataset A and Dataset C. While it would be interesting to attempt to identify birds across that gap, the shared classes of $A \cap (B \cup C) = 2$. While B and C only have a difference of a few weeks, they have a larger overlap of classes ($B \cap C = 10$) and some individuals display some change between the two datasets, with Y-MY progressing through a beak fungal infection.

As a result, I've chosen to investigate Dataset B, with the intent to eventually test the classifier on Dataset C to evaluate how it performs on classifying identities when features change over time.

III. Instance segmentation

To avoid individual birds from being associated with background features, it is necessary to mask out the background of an image of a bird so the later classifier only classifies based on bird features. To this end, Ferreira et al. 2020 used Matterport's 2018 Mask-RCNN implementation to segment out select bird species and produce masked images; I initially plan to follow Mask-RCNN, seeking other methods of instance segmentation if Mask-RCNN performs suboptimally (does not mask out the nozzle or capture the beak correctly). Following Ferrriera's methods, I plan to:

- Gather 200 images of the dataset and hand-annotate them in VGG Image Annotator.
- Split the 200 images and their annotations into a 80/20 training/validation set.
- Train Mask-RCNN given the new training and validation set using keras EarlyStopping and ReduceLROnPlateau on val_loss.
- If $val\_loss > 0.1$, or if Mask-RCNN is evaluated on 20 images and found it consistently misses the beak tip or includes the nozzle, gather another 100 images and repeat the process.
- If after 500 images annotated Mask-RCNN continues to underperform, and if time allows, look at alternative instance segmentation methods, such as YOLO or PointRend.

IV. Image pre-processing

Masked birds will have a variety of image transformations applied as a method of pre-processing, following the transformations described in Ferreira et al.'s 2020 Individual-BirdID codebase. Transformations include Gaussian noise, resizing, Gaussian blur and motion blur. These image transformations will be saved locally and will help the CNN classifier generalize a kākā's features.

In addition to these local transformations, I will implement image augmentation from keras, with more augmentations given to birds with comparatively few photos to avoid class imbalance. I will aim to augment photos until each class reaches 1000 images, in line with the number of images per class that Ferreira aimed for.

V. Genetic CNN Classifier

Hand-built CNN architectures often need expert knowledge to craft an architecture that performs optimally on a given task. Genetic algorithms can generate CNN architectures that are competitive with state-of-the-art CNN architectures [5] [6]; genetic algorithm search also avoids the higher compute costs of algorithm searching that grid search or reinforcement learning may entail [5].

I will utilise an existing open-source Evolutionary Neural Algorithm Search (ENAS) codebase, such as EEEA-Net [6] or MFENAS [7]. While different ENAS codebases will differ in their exact approaches, most genetic search algorithms operate on the same basic principles:

- Creating an initial generation of N_n models with random 'genotype' that describes its hyperparameters (layer count and type, kernel size, neuron count, type of activation function, etc.)
- Repeat the following N_g times or until fitness plateaus or reaches a target:
- Train models and evaluate their performance as 'fitness' (often a function that combines validation loss, accuracy, and a measure of model simplicity)
- Creating a new generation based on two models with the best fitness by combining their 'genotypes' of hyperparameters at random, often with a chance of mutation in a hyperparameter

EEEA-Net is particularly suitable due to its use of Early Exit Population Initialisation (EE-PI). EE-PI discards any models in the first generation that have too many parameters and associated high compute costs, thus biasing future generations towards model simplicity and helping them adhere to the law of parsimony.EEEA-Net

The most expensive step of ENAS is training the models [7]. In the interest of conserving compute, I will use a smaller subset of Dataset B, augmented for equal class numbers, for the genetic algorithm. I will then train the final 'winning' model on the full Dataset B and evaluate its accuracy, error rate, and model structure. I will additionally limit each generation to $N_n = 8$ individuals, and stop training each individual after 10 epochs.

Assuming each CNN takes 20 minutes to train, each generation will take 160 minutes. Presuming I aim for 8-hour blocks of training per day, I could achieve 20 generations in 7 days.

After evolving architectures over $N_g = 20$ generations (or until fitness plateaus), I will evaluate the best model's performance on Dataset B, testing its I will also evaluate it on Dataset C to test generalization across temporally distinct image sets.

VI. Expected Outcomes

As the quality of the final classifier will depend on the quality of the instance segmentation method, there are two separate expected outcomes.

A. Instance segmentation

It's important to avoid information that is irrelevant to the kākā's identity - such as the feeder nozzle - being mistaken as a feature of a kākā. As such, the instance segmentation method needs to reliably mask out the feeder nozzle.

As it's currently hypothesized that individual kākā identification is heavily dependent on capturing a kākā's beak features [1], it's also important that the instance segmentation method is capable of reliably capturing the kākā's beak in its entirety.

Failure to do so on either count may have a negative impact on the accuracy of the CNN model, which uses the masked outputs of the instance segmentation model as its dataset.

B. Genetic CNN

The pipeline should produce a final CNN that is capable of successfully identifying individual kākā in an unseen evaluation set with an average accuracy of $accuracy > 50\%$ across classes.

VII. Potential limitations and future work

While I currently have a working version of Mask-RCNN that follows Ferreira's implementation with 500 images, I've found that Mask-RCNN often misses the tip of the beak or includes the feeder nozzle in the background as part of the bird instance. This is a known issue with Mask-RCNN's low resolution masks that later models have attempted to correct [8], such as the 2020 implementation PointRend. If time allows, I will explore more modern instance segmentation implementations such as PointRend or YOLO.

VIII. Statement

I verify that I have written this report in its entirety without assistance. Tools used for this project include Visual Studio Code [9], Tensorflow [10], Jupyter Notebook [11], VGG Image Annotator [12], akTwelve's tensorflow2 upgrade [13] of Matterport's implementation [14] of Mask-RCNN [15], Mask-RCNN Custom.py by soumyaiitkgp [16], and Bird-IndividualID by Ferreira et al. 2020 [3]. A link to my project and an associated progress journal can be found here.

References

[1] F. O'Sullivan, K.-R. Escott, R. C. Shaw, and A. Lensen, "Feature-based image matching for identifying individual kākā," 2023.

[2] P. Maddigan, O. Ehrhardt, A. Lensen, and R. C. Shaw, "Re-identification of individual kākā: An explainable dino-based model," 2024.

[3] F. R. H. B. B. J. P. R. D. R. F. R. C. André C. Ferreira, Liliana R. Silva and C. Doutrelant, "Deep learning-based methods for individual recognition in small birds," *Methods in Ecology and Evolution*, 2020.

[4] O. Ehrhardt, R. Shaw, and A. Lensen, "Facial recognition of kākā at zealandia: Summer research project – 2022/23," 2023.

[5] S. S. Masanori Suganuma and T. Nagao, “A genetic programming approach to designing convolutional neural network architectures,” *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 497–504, 2017.

[6] C. Termritthikun, Y. Jamtsho, J. Ieamsaard, P. Muneesawang, and I. Lee, “Eeea-net: An early exit evolutionary neural architecture search,” *Engineering Applications of Artificial Intelligence*, vol. 104, p. 104397, Sept. 2021.

[7] S. Yang, Y. Tian, X. Xiang, S. Peng, and X. Zhang, “Accelerating evolutionary neural architecture search via multi-fidelity evaluation,” 2021.

[8] A. Kirillov, Y. Wu, K. He, and R. Girshick, “Pointrend: Image segmentation as rendering,” 2020.

[9] Microsoft, “Visual studio code.” https://code.visualstudio.com/. Accessed: 2025-07-30.

[10] Google Brain Team, “Tensorflow.” https://www.tensorflow.org/. Accessed: 2025-07-30.

[11] P. Jupyter, “Jupyter notebook.” https://jupyter.org/. Accessed: 2025-07-30.

[12] O. V. G. Group, “Vgg image annotator (via) v1.0.6.” https://www.robots.ox.ac.uk/~vgg/software/via/via-1.0.6.html. Accessed: 2025-07-30.

[13] akTwelve, “Tensorflow 2 upgrade of matterport’s mask-rcnn implementation.” https://github.com/akTwelve/Mask_RCNN, 2020. Accessed: 2025-07-30.

[14] W. Abdulla, “Mask r-cnn for object detection and instance segmentation.” https://github.com/matterport/Mask_RCNN, 2017. Accessed: 2025-07-30.

[15] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask r-cnn,” 2018.

[16] soumyaiitkgp, “Mask-rcnn custom.py sample script.” https://github.com/soumyaiitkgp/Custom_MaskRCNN/blob/master/samples/custom/custom.py, 2020. Accessed: 2025-07-30.